

An Introduction to Graph Neural Networks in Computer Vision: A Practical Perspective

Bárbara Caroline Benato, Lucas Pascotti Valem, Gabriel Maia Brito,
Ian de Holanda Cavalcanti Bezerra, Rafael Mendonça Duarte, and Luis Gustavo Nonato
Institute of Mathematics and Computer Science (ICMC), University of São Paulo (USP), São Carlos, Brazil
Email: barbarabenato@gmail.com, lucas@icmc.usp.br, gabrielmaiab@usp.br,
idhcb.ian@usp.br, rmduarte@usp.br, gnonato@icmc.usp.br

Abstract—While deep learning models such as Convolutional Neural Networks (CNNs) and Vision Transformers (ViT) have achieved remarkable success in computer vision, they are inherently limited to processing data with fixed and regular structures, such as 2D grids. However, Graph Neural Networks (GNNs) extend the convolutions to non-Euclidean domains, allowing them to process data structured as graphs. This capability enables GNNs to exploit complex relationships beyond the fixed spatial organization of pixels. Another important aspect is that GNNs are particularly well-suited for semi-supervised learning scenarios, especially those with few labeled data points. This is a significant advantage in computer vision, where obtaining large volumes of labeled data for training deep learning models is usually necessary. This paper provides a practical overview of GNNs and their application to visual data, with a focus on image classification, retrieval, segmentation, and detection. We begin with foundational concepts and representative GNN models. Building on this foundation, we examine how visual graphs can be constructed from deep features and how GNNs can be applied to different computer vision tasks. Finally, we highlight current trends, key challenges, and promising directions for future research on GNNs in computer vision.

I. INTRODUCTION

With the rise of deep learning, computer vision tasks have been predominantly tackled using Convolutional Neural Networks (CNNs) [1] and, more recently, Vision Transformers (ViTs) [2]. The field has seen a proliferation of increasingly complex architectures, which often hamper interpretability. Despite their success, standard CNNs constrain image representation to Euclidean space. Alternatively, Graph Neural Networks (GNNs) [3], [4] extend the concept of convolutions to non-Euclidean domains [5], allowing for the efficient generation of node representations that incorporate both individual node features and the structure of their local neighborhoods.

Moreover, GNN-based methods are versatile and can support a wide range of tasks, including node classification, edge prediction, and graph-level tasks such as graph isomorphism and classification. This flexibility has been explored in various computer vision applications, such as image classification, retrieval, and segmentation [6], [7]. Nevertheless, their use in vision remains relatively limited compared to other domains. A major reason for this is the difficulty in constructing appropriate graph representations for image data, which typically do not come with an inherent graph structure.

While recent advances have highlighted the potential of transformers in graph-based learning [8], traditional GNN-

based methods remain highly competitive and often perform on par with state-of-the-art computer vision techniques across a variety of tasks [9]. However, their performance is highly dependent on the quality of the underlying graph structure. Since GNNs are particularly sensitive to the organization of node neighborhoods, poorly constructed graphs can impede the effective propagation of information, ultimately reducing model performance.

This paper focuses on the integration of CNNs and GNNs, more specifically, graph convolutional networks (GCNs), for computer vision tasks. It is designed for an audience with little to no prior experience in GNNs, but with an interest in deepening their understanding of the topic. We start with a gentle introduction to GNNs from foundation concepts, building up the multilayer perceptron formulation to describe message-passing, attention mechanisms, and loss functions, for GCN and Graph Auto-Encoder Networks (GAE). We explore how to construct visual graphs from deep features and apply GNNs to structured visual tasks: image classification, retrieval, and segmentation.

The discussion draws on publicly available datasets and pretrained feature extractors based on CNNs and ViTs. In addition, we present GCNVis, a web-based visual tool ¹ that supports the interactive visualization of input graphs, allowing users to inspect their components and incorporate domain insights. Finally, we discuss key challenges and promising directions for future research on GNNs in computer vision.

II. FOUNDATIONS OF GRAPH NEURAL NETWORKS

A. Graph definition

A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ can be defined as a set of nodes $\mathcal{V}$, also called vertices, and a set of edges $\mathcal{E}$ connecting pairs of nodes $u, v \in \mathcal{V}$, denoted as $(u, v) \in \mathcal{E}$. Graph nodes can bear feature vectors $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, with $\mathbf{x}_i \in \mathbb{R}^d$, d corresponding to the number of features and n the number of nodes in V , denoted as $n = |V|$. Furthermore, a set of labels $\mathcal{Y} = \{y_1, y_2, \dots, y_n\}$, one associated with each node, can be provided, with $y_i \in \{1, 2, \dots, c\}$, where c is the number of classes for a classification task.

If a graph is an *undirected graph*, for each $(u, v) \in \mathcal{E}$ there is also $(v, u) \in \mathcal{E}$. For *directed graphs*, the direction of an edge (u, v) matters, and it does not directly imply an edge (v, u) . Moreover, graphs can have *weights* $\mathcal{W}$ associated with

the edges. In our context, the weights $w \in \mathcal{W}$ are real numbers in the interval $[0, 1]$ quantifying the distance, importance, or strength of the connection between nodes.

Usually, graphs are represented by an adjacency matrix A of size $|\mathcal{V}| \times |\mathcal{V}|$, in which rows and columns correspond to nodes. An entry $A[u, v]$ in A is equal to 1 if there is an edge (u, v) connecting the nodes u and v , and 0 otherwise, that is, $A[u, v] = 1$ if $(u, v) \in \mathcal{E}$ and $A[u, v] = 0$ if $(u, v) \notin \mathcal{E}$, respectively. When weights are associated with the edges, $A[u, v] = 1$ is replaced by $A[u, v] = w(u, v)$, where $w(u, v)$ is the weight associated with the edge (u, v) .

B. GNN basics

A GNN can be interpreted as a multilayer perceptron neural network that explores a graph structure. The main intuition behind a GNN is the *neural message-passing* [10] mechanism in which information among nodes is exchanged and updated throughout the neural network layers. In other words, neighboring features are passed to the source node through the edges that connect them. Features are first combined by an *aggregation* function to compose shared information. After that, aggregated features are *updated* in a way that the source node retains information from all its neighbors. All nodes in a graph are processed this way in each GNN layer. The updated features are used as the input to the next layer, and this processing finishes after a few layers, resulting in learned feature vectors that capture complex relationships among neighboring nodes.

C. Main GNN models

By choosing different combinations of *aggregation* and *update* functions, different GNNs models can be generated. Among them, convolutional networks [5], graph auto-encoder [11], and graph-attention networks [12] are examples of the most known in the literature, which are explained as follows:

- **Graph Convolutional Network (GCN):** When computing the message-passing step, a node with many neighbors may have higher feature values (due to the aggregate function) than a node with only a few neighbors. To solve this, GNNs are improved with normalization based on the neighborhood sizes. In GCNs, we first calculate the symmetric normalization [5] of a neighborhood. This normalization strategy has led GCNs to be a good baseline for graph network models [9], [13].
- **Graph Auto-Encoder (GAE):** As a representation learning strategy, autoencoders are used to create low-dimensional representations of graph nodes in an unsupervised fashion. Graph Auto-Encoder (GAE) [11] uses an encoder-decoder framework to learn a representation of nodes and their neighbors in the adjacency matrix A . A GCN is usually used as the encoder step, generating encoded representations for all nodes in the graph. For the decoder step, a similarity function is used to compute the reconstructed representation. After that, GAE is trained by the mean-squared error computed between the reconstructed adjacency matrix A' and the original A .
- **Graph Attention Network (GAT):** In the *aggregation* step mentioned before, neighbors have equal weights of

importance when aggregating feature information. It is not possible to calculate the importance of each neighbor to the source node. To solve this, the Graph Attention Network (GAT) [12] is proposed in the literature to consider an *attention weight* to each neighbor message-passing.

III. BUILDING GRAPHS FROM VISUAL DATA

Given that images are not directly mapped as a graph, defining the image representation is an important step of the pipeline when using GNNs in computer vision.

From the literature, it is possible to notice different approaches to build graph structures from image data. While many reviews did not discuss such approaches for building graphs [14]–[16], others presented some limitations. In [6], pixel-based, superpixel-based, and object-based graphs are discussed. In [17], labels, pixels, regions, and other levels are discussed for different computer vision applications. Neither review considers deep features to be a category of graph structure for image data.

For this reason, we discuss the **three** main approaches used in computer vision applications to build graph structures as follows:

- **Pixel-level:** Each image pixel is defined as a node in the graph, while the neighborhood of each pixel determines the other pixels with an edge. Pixels are only for small image datasets in classification [18], [19], but mainly in segmentation [20], [21] and object detection tasks.
- **Region-level:** Since having each pixel as a node can increase the size of the graph, strategies have used superpixels or image patches as nodes in the graph. Edges connect neighbors of each region. In this case, an example shows applications to classification [22], while mostly are concerned to segmentation [23]–[25] and object detection [26].
- **Feature-level:** Still, some regions may have redundant pixel values and/or lack shape and texture information. For this reason, studies have explored deep feature embeddings extracted from CNNs/ViTs to be nodes in the graph. Edges are based on some similarity distance between features, usually computed by k-NN or cosine similarity. For CNN feature level, many studies have focused on image classification [27], [28], segmentation [29], [30], and object detection [31].

In computer vision, different studies have compared purely deep learning methods with deep learning-based GNNs [15], [17], [32], [33]. As a result, they showed that exploring the advantages of feature extraction by deep learning and non-Euclidean relationships by GNNs have surpassed methods purely based on deep learning methods.

IV. DATASETS AND DEEP FEATURE EXTRACTION

A. Datasets

- **Flowers17** [34]: consists of 1,360 images evenly distributed across 17 flower categories. Each class contains 80 images captured under varying backgrounds and viewpoints, offering moderate intra-class variability.

- **Corel5k** [35]: consists of 5,000 natural images, spanning 50 categories such as animals, landscapes, and scenes.
- **Pets** [36]: comprises 7,390 images from 37 pet classes, including high variability in pose, lighting, and background.
- **PASCAL VOC 2012** [37]: contains 17,125 images of real-life scenes of 20 object categories, including bounding boxes, object classes, and contours.
- **MS-COCO** [38]: is composed of 328k images of everyday scenes with 80 object categories for object detection, segmentation, and captioning dataset.

B. Deep feature extraction

For each dataset, we employ three deep learning models pretrained on the ImageNet dataset:

- **ResNet152** [39]: a deep residual CNN. We used the 2048-dimensional output from the final convolutional stage.
- **SENet154** [40]: a ResNet variant with channel-wise attention. Features were extracted from the final SE block (2048 dimensions).
- **ViT-Base** [41]: a transformer model operating on patch sequences. We used the 768-dimensional class token from the last encoder layer.

V. APPLICATIONS

In computer vision, GNNs are mainly used to identify the context of a group of data by creating a graph that connects image segments to one another, or even full images, based on similarity. This connection provides a new way to look at the dataset and see the relationship between each part of the data. In this section, we cover two practical applications: (i) image classification and retrieval; and (ii) image segmentation tasks.

A. Image Classification and Retrieval

We consider a practical application of GNNs to a semi-supervised image classification scenario. The goal is to consider labeled and unlabeled image examples in a dataset to train a classifier. Each image in the dataset refers to a node in the graph.

1) *GNN models*: In addition to the GCN model proposed by Kipf et al. [5], the following representative variants are also discussed:

- **SGC** [42]: The Simple Graph Convolution model eliminates intermediate nonlinearities and merges multiple weight matrices into a single linear transformation, which is applied after propagating node features through the adjacency matrix for k steps.
- **APNP** [43]: The Approximate Personalized Propagation of Neural Predictions approach separates feature transformation and propagation. It first generates node embeddings using a multilayer perceptron and then applies a personalized PageRank scheme to propagate them through the graph.
- **ARMA** [44]: This model applies AutoRegressive Moving Average filters to perform graph convolutions as recursive propagation steps. By stacking multiple ARMA layers, it enables deep signal filtering while maintaining stable frequency responses.

2) *Workflow*: The steps for the process are in Figure 1. Each step is detailed as follows:

- 1) **Feature Extraction**: The GCN cannot operate directly on raw images; we must first represent each image in the dataset as a numerical feature vector. This vector should capture the visual characteristics of the image. For this task, we utilize descriptors mentioned in Sec. IV-B.
- 2) **Compute k-NN graph**: To build the graph, a ranked list for every image in the dataset is created. This list sorts all other images based on their visual similarity to the reference image. The quantification of the similarity is provided by the Euclidean distance between their respective feature vectors. Because of this, smaller distances imply greater similarity, placing the image at a lower index in the ranked list. This process creates an $N \times N$ matrix, where N is the size of the dataset and each row represents the ranked list of an image. With the ranked lists established, the neighborhood of an image is the set containing the first k images from its ranked list. After the ranked lists, the graph is created. The strategies to define the edges based on the neighborhoods are variable.

- **k -Nearest Neighbors (k NN)**: Edges are formed by connecting each image to its k most similar neighbors.
- **Reciprocal k -Nearest Neighbors (Rec- k NN)**: Edges are included only between pairs of images that are mutually among each other's k nearest neighbors. This method, unlike only k NN, creates undirected edges, making the connections stronger and consequently more meaningful, although the graph is sparser.

- 3) **Refinement and Predictions with GCN**: The core of the classification resides in this step. The principal function of the GCN, a type of GNN, is its capability to analyze not only the image itself but also the relationships within its neighborhood (i.e., its connections). Its functioning is based on the propagation of information that updates the feature vectors in layers. In each layer, an image aggregates information from its neighbors to refine its own representation. Finally, the GCN outputs a vector for each image containing the class probabilities, which are derived from the final embeddings, and we can make the image classification.

- 4) **Building Refined Ranked Lists**: With the embeddings generated by the GCN, the process described in step 2 is performed again. However, instead of using the initial features, the refined embeddings are used. Ranked lists tend to be more coherent and accurate after utilizing this method since embeddings carry more essential information. Figure 2 represents the ranked lists before and after the refinement, showing the improved results. This process is only utilized if the final objective includes an image retrieval task.

After creating the image representations with the feature extraction models (Sec. IV-B), we developed GCNVis to visualize the image graphs used as input to the GCN. Figure 3

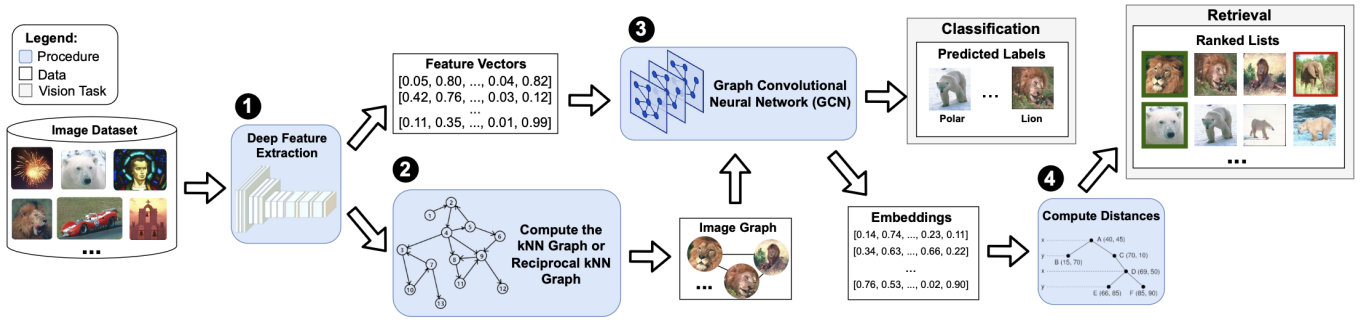


Fig. 1: Workflow for using GCNs in image classification and retrieval tasks based on Manifold-GCN [45].

Ranked List from Input Features (AlexNet):



Ranked List from GCN Output Embeddings:



Fig. 2: Ranked lists for a query image (blue), showing the most similar results. Incorrect matches are in red. The first list uses deep features; the second uses GCN embeddings.

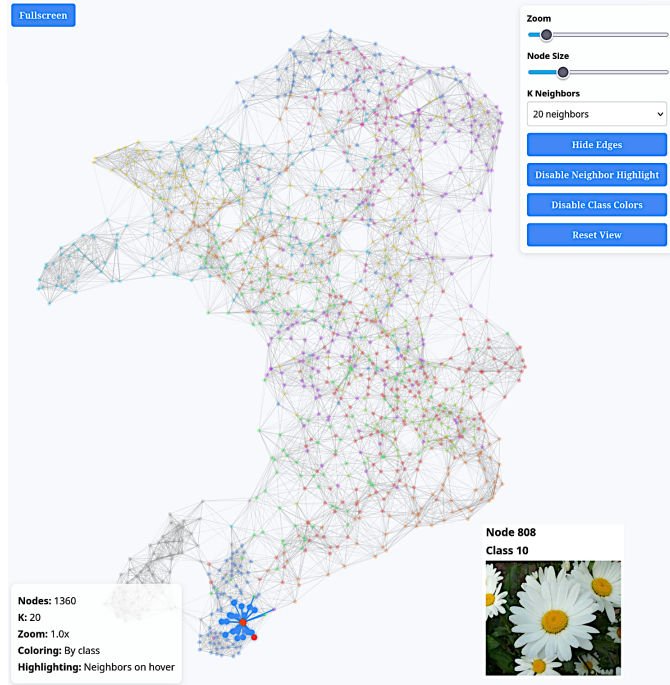


Fig. 3: Web tool for visualizing image graphs used as GCN input. Nodes are colored by class, and the selected image is displayed in the bottom right corner on hover. The neighbors of the selected node are highlighted.

and edges the similarity among images. The node colors represent classes. In the web tool, it is possible to visualize graph information: number of nodes, k-NN parameters, and the type of coloring. Also, when hovering over a node in the visualization, its respective image is presented, and neighbors are highlighted in the graph. Different aspects of the graph can be defined as the number of neighbors for k-NN, node size in the visualization, and zoom. This visualization allows for the comparison of different feature extraction algorithms, parameters, and datasets for classification purposes.

B. Image Segmentation and Detection

The application of GNNs to image segmentation and detection, particularly in an unsupervised manner, is an emerging and innovative area of research. A central motivation for using GNNs in image segmentation is to address the challenges associated with supervised learning, particularly the need for large volumes of annotated data. For this task, we use PASCAL VOC 2012 dataset.

1) *GNN models*: This section discusses two representative graph-based frameworks for fully unsupervised image segmentation. These frameworks combine strong transformer-derived features with lightweight GNNs trained using classical graph partitioning objectives:

- **DeepCut**² [46]: is a self-supervised method designed for fully unsupervised object localization, object segmentation, and semantic part segmentation. DeepCut [46] adopts a test-time training strategy in which the model is optimized individually for each image. For object localization and segmentation, weights are reset between images and training runs for a few epochs (e.g., 10), while for semantic part segmentation, weights are retained across images and training is longer (e.g., 100 epochs).
- **UnSegGNet**³ [31]: a method that can be viewed as a direct modification of DeepCut's design. One of the key differences is that it trains the GNN once on the entire training set and reuses it for all test images, significantly reducing inference time and enabling evaluation on larger datasets.

The two frameworks are complementary: DeepCut is more suitable when per-image adaptation is important (e.g., seman-

²DeepCut: <https://github.com/SAMPL-Weizmann/DeepCut>

³UnSegGNet: <https://github.com/ksgr5566/unseggnet>

illustrates the graph model with nodes representing images

tic part segmentation), while UnSegGNet is preferable for fast batch inference or when balanced clustering is desired.

2) *Workflow*: Figure 4 illustrates the overall pipeline for image segmentation. Steps 3, 4, and 5 were grouped to simplify the visualization and reduce the overall length of the figure. Each step is detailed as follows:

- 1) **Patch-wise Feature Extraction from a Pre-trained Vision Transformer (ViT)**: The image is divided into multiple patches p , and a feature vector is extracted for each patch using a pre-trained Vision Transformer, specifically the DINO-ViT-S/8 model. The representation is obtained from the key token of the transformer’s final layer.
- 2) **Graph Construction**: A weighted graph representation $G = (V, E)$ is constructed for the image, where each node $v \in V$ represents an image patch. The weights (affinities) between these nodes are derived from the patch-wise correlation matrix of the extracted deep features.
- 3) **Correlation Clustering (CC) Objective**: For the CC objective, both positive and negative affinities are utilized. DeepCut uses a k -sensitivity hyperparameter α to adapt the cluster choosing process and control the number of clusters without pre-defining k .
- 4) **Self-supervised GNN Training**: The constructed adjacency matrix W (pairwise affinities) and the raw deep features f (node features) are both used as input to a single-layer GNN that updates patch embeddings through message passing. The GCN output is then passed to a softmax Multi-layer Perceptron (MLP) to produce the cluster assignment matrix S , enabling correlation clustering to determine the number of clusters without any predefined k . GNN is optimized with either the relaxed Normalised-Cut loss or the Correlation-Clustering loss, selectable at run time.
- 5) **Two-stage Clustering for Part-level Segmentation**: An initial clustering is performed with $k=2$ to coarsely separate background and foreground regions. The foreground region is then clustered again using a higher k (e.g., $k=4$) to capture finer, part-level structures within the object.
- 6) **Generating the Final Segmentation Map**: After the GNN is trained and the cluster assignment matrix S is obtained, the final segmentation map is generated by taking the argmax of S for each patch. This assigns each image patch to its most likely cluster, and the result is then unflattened to obtain the segmentation map.

VI. CONCLUSION AND FUTURE DIRECTIONS

This paper presented the foundations and applications of graph neural networks in computer vision. We reviewed representative GNN architectures and the construction of visual graphs at the pixel, region, and feature levels. We also discussed GNN-based pipelines for semi-supervised image classification, content-based image retrieval, and unsupervised segmentation and detection, including the DeepCut and UnSegGNet frameworks. In addition, GCNVis was presented as

an interactive tool for inspecting graph structures used as input to GCNs.

Across these applications, a recurring trend is the combination of representations extracted by pretrained CNNs or ViTs with graph-based propagation. This combination enables models to exploit relationships that are not explicitly represented by regular image grids. However, because images do not provide an inherent graph topology and GCNs are sensitive to neighborhood organization, inadequate graph construction can hinder information propagation and reduce model effectiveness.

Promising directions for future work include adaptive and task-aware graph construction, joint optimization of visual representations and graph topology, and graph structures that remain robust across datasets and applications. Further investigation of interpretable representations and label-efficient learning may also broaden the applicability of GNNs to computer vision tasks.

ACKNOWLEDGEMENTS

We thank the São Paulo Research Foundation – FAPESP (grants #2025/10602-5, #2026/13387-0, #2026/16921-8), the University of São Paulo (PRPI Ordinance No. 1032, New Faculty Support Program), and the Institute of Mathematics and Computer Science (ICMC) for financial support.

REFERENCES

- [1] L. Y. B. L., B. Y., and H. P., “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.
- [2] J.-B. Cordonnier, A. Loukas, and M. Jaggi, “On the relationship between self-attention and convolutional layers,” in *ICLR*, 2020.
- [3] M. Gori, G. Monfardini, and F. Scarselli, “A new model for learning in graph domains,” in *IJCNN*, vol. 2, 2005, pp. 729–734.
- [4] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *5th International Conference on Learning Representations, ICLR*, 2017.
- [6] U. Nazir, H. Wang, and M. Taj, “Survey of image based graph neural networks,” *arXiv preprint arXiv:2106.06307*, 2021.
- [7] U. A. Bhatti, H. Tang, G. Wu, S. Marjan, and A. Hussain, “Deep learning with graph convolutional networks: An overview and latest applications in computational intelligence,” *International Journal of Intelligent Systems*, vol. 2023, no. 1, p. 8342104, 2023.
- [8] V. P. Dwivedi and X. Bresson, “A generalization of transformer networks to graphs,” *arXiv preprint arXiv:2012.09699*, 2020.
- [9] Y. Luo, L. Shi, and X.-M. Wu, “Classic GNNs are strong baselines: Reassessing GNNs for node classification,” in *The 38th Conf. on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [10] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *International conference on machine learning*. Pmlr, 2017, pp. 1263–1272.
- [11] T. N. Kipf and M. Welling, “Variational graph auto-encoders,” *NIPS Bayesian Deep Learning Workshop*, 2016.
- [12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [13] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.
- [14] M. Krzywdka, S. Łukasik, and A. H. Gandomi, “Graph neural networks in computer vision - architectures, datasets and common approaches,” in *International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–10.

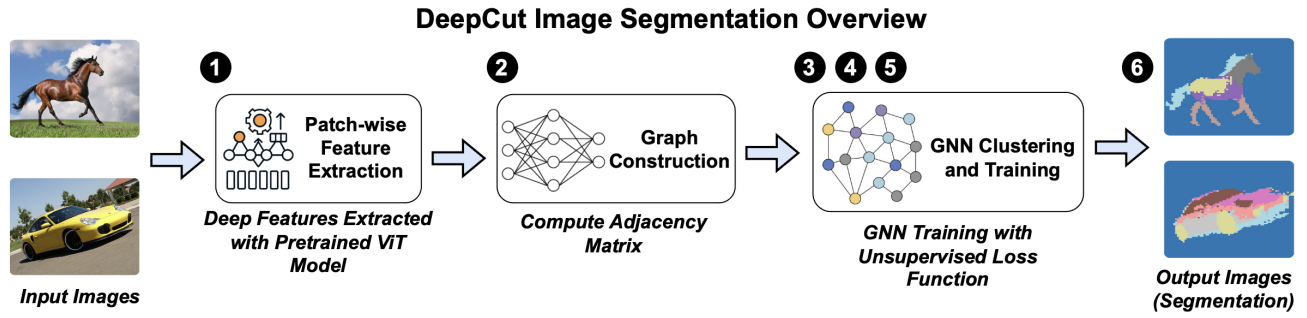


Fig. 4: Overview of steps for unsupervised image segmentation with GNNs using the DeepCut framework [46].

- [15] C. Chen, Y. Wu, Q. Dai, H.-Y. Zhou, M. Xu, S. Yang, X. Han, and Y. Yu, "A survey on graph neural networks and graph transformers in computer vision: A task-oriented perspective," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10297–10318, 2024.
- [16] Z. Jia, C. Wang, Y. Wang, X. Gao, B. Li, L. Yin, and H. Chen, "Recent research progress of graph neural networks in computer vision," *Electronics*, vol. 14, no. 9, 2025.
- [17] L. Jiao, J. Chen, F. Liu, S. Yang, C. You, X. Liu, L. Li, and B. Hou, "Graph representation learning meets computer vision: A survey," *IEEE Transactions on Artificial Intelligence*, vol. 4, no. 1, pp. 2–22, 2023.
- [18] M. Tan, Q. Shi, A. van den Hengel, C. Shen, J. Gao, F. Hu, and Z. Zhang, "Learning graph structure for multi-label image classification via clique generation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 4100–4109.
- [19] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," *Advances in neural information processing systems*, vol. 29, 2016.
- [20] L. Wang, Y. Huang, Y. Hou, S. Zhang, and J. Shan, "Graph attention convolution for point cloud semantic segmentation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10288–10297.
- [21] Y. Lu, Y. Chen, D. Zhao, and J. Chen, "Graph-fcn for image semantic segmentation," in *International symposium on neural networks*. Springer, 2019, pp. 97–105.
- [22] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 1, pp. 597–612, 2021.
- [23] Z. Khatun, H. Jónsson, M. Tsirilaki, N. Maffulli, F. Oliva, P. Daval, F. Tortorella, and P. Gargiulo, "Beyond pixel: Superpixel-based mri segmentation through traditional machine learning and graph convolutional network," *Computer Methods and Programs in Biomedicine*, vol. 256, p. 108398, 2024.
- [24] C. Wu, Z. Feng, H. Zhang, and H. Yan, "Graph neural network and superpixel based brain tissue segmentation," in *International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 01–08.
- [25] A. Jain, D. H. Laidlaw, P. Bajcsy, and R. Singh, "Memory-efficient semantic segmentation of large microscopy images using graph-based neural networks," *Microscopy*, vol. 73, no. 3, pp. 275–286, 10 2023.
- [26] X. Li, Y. Li, C. Shen, A. Dick, and A. V. D. Hengel, "Contextual hypergraph modeling for salient object detection," in *2013 IEEE International Conference on Computer Vision*, 2013, pp. 3328–3335.
- [27] J. Zhao, K. Yan, Y. Zhao, X. Guo, F. Huang, and J. Li, "Transformer-based dual relation graph for multi-label image recognition," in *Proc. of the IEEE/CVF international conference on computer vision*, 2021, pp. 163–172.
- [28] S. M. T. U. Raju, M. M. Islam, M. R. Haque, H. Altafheri, and F. Karray, "Gnn-vitcap: Gnn-enhanced multiple instance learning with vision transformers for whole slide image classification and captioning," 2025. [Online]. Available: <https://arxiv.org/abs/2507.07006>
- [29] H. Hu, D. Ji, W. Gan, S. Bai, W. Wu, and J. Yan, "Class-wise dynamic graph convolution for semantic segmentation," in *European Conference on Computer Vision*. Springer, 2020, pp. 1–17.
- [30] G.-S. Xie, J. Liu, H. Xiong, and L. Shao, "Scale-aware graph neural network for few-shot semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 5475–5484.
- [31] K. S. G. Reddy, B. Saran, M. Adityaja, S. J. Shigwan, and N. Kumar, "UnSegGNet: Unsupervised image segmentation using graph neural networks," *arXiv preprint arXiv:2405.06057*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.06057>
- [32] G. Nikolentzos, M. Thomas, A. R. Rivera, and M. Vazirgiannis, "Image classification using graph-based representations and graph neural networks," in *Complex Networks & Their Applications IX*, R. M. Benito, C. Cherifi, H. Cherifi, E. Moro, L. M. Rocha, and M. Sales-Pardo, Eds., 2021, pp. 142–153.
- [33] X. Liu, Y. Su, and B. Xu, "The application of graph neural network in natural language processing and computer vision," in *2021 3rd International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI)*, 2021, pp. 708–714.
- [34] M.-E. Nilsback and A. Zisserman, "A visual vocabulary for flower classification," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, vol. 2, 2006, pp. 1447–1454.
- [35] G.-H. Liu and J.-Y. Yang, "Content-based image retrieval using color difference histogram," *Pattern Recognition*, vol. 46, no. 1, pp. 188 – 198, 2013.
- [36] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [37] M. Everingham, S. A. Eslami, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes challenge: A retrospective," *International journal of computer vision*, vol. 111, no. 1, pp. 98–136, 2015.
- [38] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision – ECCV*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds., Cham, 2014, pp. 740–755.
- [39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [40] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *2018 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [41] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.
- [42] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6861–6871.
- [43] J. Klicpera, A. Bojchevski, and S. Günnemann, "Combining neural networks with personalized pagerank for classification on graphs," in *International Conference on Learning Representations*, 2019.
- [44] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph neural networks with convolutional arma filters," *IEEE TPAMI*, pp. 1–1, 2021.
- [45] L. P. Valem, D. C. G. Pedronette, and L. J. Latecki, "Graph convolutional networks based on manifold learning for semi-supervised image classification," *Computer Vision and Image Understanding*, vol. 227, p. 103618, 2023.
- [46] A. Aflalo, S. Bagon, T. Kashti, and Y. Eldar, "Deepcut: Unsupervised segmentation using graph neural networks clustering," in *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2023, pp. 32–41.